

The Need for Being Explicit: Failed Attempts to Construct Implicit Certificates from Lattices

NINA BINDEL¹ AND SARAH MCCARTHY^{2,*}

¹*SandboxAQ, Palo Alto, CA, USA*

²*Institute for Quantum Computing, University of Waterloo, 200 University Ave W, Waterloo, ON N2L 3G1, Canada*

*Corresponding author: sarah.mccarthy@uwaterloo.ca

Global efforts such as the National Institute of Standards and Technology (NIST)'s post-quantum standardization center on cryptographic primitives like public-key encryption and signature schemes that are secure even in the presence of quantum adversaries. In addition, one must also consider efficient certificate management as new technologies like the Internet of Things and 5G wireless networks rely on them. For example, the IEEE Standard for vehicle-to-vehicle communication depends on implicit certificates. However, the only efficient construction available is over elliptic curves, and hence not quantum-secure. This paper investigates approaches to construct implicit certificate schemes from lattices, employing the NIST Round 3 signature schemes Dilithium and Falcon. We consider emulation of the existing implicit certificate scheme and proceed to more innovative techniques like combining the two schemes or pairing them with encryption. Unfortunately, we encounter problems with each design, due to recurring causes like conflicting secret key and signature sizes, unique sampler requirements and the rigidity of the parameter sets. By explaining each of these issues, this paper will hopefully spark ideas for more successful constructions.

Keywords: implicit certificates; lattice-based cryptography; post-quantum cryptography constrained environments

Received 31 January 2022; Revised 12 July 2022; Editorial Decision 10 August 2022

Handling editor: Tomer Ashur

1. INTRODUCTION

To maintain security during the quantum computing era, public-key cryptography systems will likely be upgraded to safeguard them against quantum attackers, i.e. to use Post-Quantum (PQ) algorithms. Whilst there are significant research and standardization activities taking place, for example the National Institute of Standards and Technology (NIST)'s efforts to select PQ Key Encapsulation Mechanisms (KEMs), Public Key Encryption (PKE) schemes and Digital Signature Schemes (DSS)s to be used in the new SP 800-56 and FIPS 186-4 standards [1], exploration into how these algorithms are compatible with various real world applications is still immature. For instance, many applications require cryptographic algorithms with more advanced functionality than KEMs, PKEs and DSSs offer. Many of these advanced

algorithms have yet to be realized in the PQ setting and evaluated in practice.

One such advanced construction is that of *implicit certificates*. As explained in due course, implicit certificates aim to reduce the large overheads associated with certificate management. To date, implicit certificates have been instantiated in Elliptic Curve Cryptography (ECC), but no PQ variants have yet been proposed. Within the five main PQ families – lattice-based, multi-variate, isogeny-based, code-based and cryptography based on symmetric primitives such as hash functions, lattices look to be the most promising in terms of offering more advanced constructions, as other sophisticated primitives such as identity-based [2, 3] and homomorphic encryption [4, 5] schemes have already been constructed from them, and are not known to have been constructed from any other of the PQ families.

In this research, we tried and failed to construct implicit certificates from the lattice-based signature schemes Falcon [6] and Dilithium [7], which are algorithms recently selected for standardization in NIST's on-going PQ standardization process [8]. This paper will explain why our constructions fail and what lessons we learned, hoping to inspire a discovery into a secure implicit certificate algorithm that is PQ secure.

Explicit vs implicit certificates. Consider a traditional Public Key Infrastructure (PKI), where the (trusted) Certificate Authority (CA) binds a user's public key to their identity. They do so by signing the user's public key (usually without knowing the user's secret key) and constructing a certificate comprised of the user's public key and identity, the (CA)'s signature, and some metadata such as the (CA)'s credentials and the certificate validity period. This is an *explicit certificate*, as any other third party can verify the (CA)'s signature using the (CA)'s public key and in doing so, knows the user public key belongs to the specified identity and that they can trust this public key. For an explicit certificate, one can see its size will be at least that of the public key and signature. The primary intention of an *implicit* certificate is to reduce this overhead, which is particularly desirable in resource constrained environments or where communication time is of the essence. An implicit certificate scheme replaces the public key and signature with a *reconstruction value* on the certificate associated to the user. Any third party can use the (CA)'s public key to reconstruct the user's public key, and in doing so, it is *implied* that the key has been validated by the (CA). As such the user's public key and the CA's signature are given *implicitly*.

Security requirements of implicit certificates Following the security model of [9], we present the existential unforgeability security notion which our proposed constructions should fulfill. It is important to note, however, since all our constructions are 'failed' that we do not provide security reductions for any of them.

DEFINITION 1.1. *Let $\mathcal{P}$ be an implicit certificate scheme. An adversary $\mathcal{A}$ targeting $\mathcal{P}$ is considered successful if they are able to forge an implicit certificate and its corresponding private key. This may be in one of two scenarios:*

- i) *the attack targets the CA, whereby $\mathcal{A}$ forges an implicit certificate not generated by the CA (potentially knowing the associated private key).*
- ii) *the attack targets the user by obtaining the implicit certificate that was generated by the CA in response to a request of the user, and forges a user's signature.*

A scheme $\mathcal{P}$ is considered (τ, ϵ) -secure if there does not exist such an adversary $\mathcal{A}$ that runs in time at most τ and succeeds with probability at least ϵ in either of the scenarios.

A brief history of implicit certificates. One of the earliest instantiations of implicit certificates in the literature was in [10]. This idea was further developed in 2000 [11], and called

the Optimal Mail Certificate (OMC) scheme. This scheme was specifically designed to be used with the Pintsov-Vanstone (PV) signature scheme and provides 40-bit security against selective forgery attacks and 80-bit against a total break [12–14]; i.e. it takes at least 2^{40} and 2^{80} operations respectively to mount a successful attack. These are stronger security notions than we consider in Definition 1.1. However, the OMC scheme has been broken by Qu and Vanstone [15], although strictly speaking this is out of the model used in [11]. They attacked the composition of OMC with the Elliptic Curve Digital Signature Algorithm (ECDSA). The attack takes the CA's public key and generates an implicit certificate and an ECDSA signature, such that the signature is valid when verified with the public key reconstructed from the implicit certificate. Kravitz [16] extended Qu's attack to a very special case of Elliptic Curve Qu-Vanstone (ECQV)-certified ECDSA in which the message signed is the ECQV certificate itself.

In 2001, Brown, Gallant and Vanstone [9] describe a modified version of the OMC implicit certificate scheme (upon which we base Definition 1.1), which has become known as the ECQV scheme. They give a definition of security for an implicit certificate scheme, and provide a proof of security for the scheme in the Random Oracle Model (ROM). Their security definition does not address the arbitrary composition of the scheme with other schemes. Rather, the threat model addresses an attacker who forges an implicit certificate together with the private key.

In 2007, Boldyreva, Fischlin, Palacio and Warinschi [17] present an ElGamal-based encryption scheme with implicit certificates. They show that if the computational Diffie-Hellman problem is hard, the scheme guarantees indistinguishability against chosen-ciphertext attacks (IND-CCA). For their security proof it suffices to use the security assurances given in [9]. Also the certified encryption in [17] can be seen as using using implicit certificates.

In 2009, Brown, Campagna and Vanstone [18] show that the composition of (ECQV) with ECDSA is secure against existential-unforgeability attacks by a passive adversary under the combination of the (ROM) and the generic group model. They build this result by revisiting the original OMC scheme. They reconstruct the aforementioned attack on the composition of OMC with ECDSA, and demonstrate why this attack fails on the composition of (ECQV) with ECDSA, also taking into account the mistake Kravitz pointed out. ECQV was standardized in 2013 [19].

More recent work [20] focuses on improving performance and reducing storage costs for ECQV by proposing extensions of the scheme, which allow users to generate their own credentials repeatedly upon receipt of a valid certificate, hence reducing the number of interactions with the CA. Similarly to this paper, they present formal security definitions of their implicit certificate schemes, relating to secrecy of the private keys, unforgeability of credentials and non-delegability of the the user's key generation function. They prove existential forgery

of ECDSA combined with their schemes against a passive adversary, following the method of [18]. Their security models assume secrecy of the private keys of the original ECQV scheme, whereas in our work we are still aiming to demonstrate the truth of this assumption in the PQ setting.

Implicit certificates in vehicle communication. Implicit certificates are heavily used in the IEEE 1609.2 and 1609.2.1 standards [21, 22] describing secure Vehicle-to-Vehicle (V2V) communication. More concretely, they use the standardized (ECQV) implicit certificate scheme [19], which is well-suited to the constraints enforced by the V2V standard [23]. According to [24], ECQV certificates are 1/3 the size of ECDSA explicit certificates, and the impact in the PQ setting can be expected to be even larger and more valuable, due to the larger sizes of explicit certificates. Parallel work [25] to this paper proposes protocols to accommodate the larger signature sizes for PQ authentication in the V2V setting, but demonstrates that implicit certificates are still desirable for enhanced security. This further motivates research into PQ implicit certificate schemes.

Constructing implicit certificates. When constructing secure implicit certificate schemes, the following construction principles must be followed intuitively¹:

Certificate size: The reconstruction value should be smaller than the CA's signature and the user's public key together, i.e. the implicit certificate should be smaller than the corresponding explicit certificate.

Computability of the user's secret key: The user needs to be able to compute their own secret key.

Secrecy of the user's secret key: Only the user should be able to compute their secret key.

Computability of the user's public key: Every party should be able to compute the user's verification key from the reconstruction value given in the certificate and the CA's public key.

Secrecy of the CA's secret key: The CA's secret must not be revealed to any other party; in particular, not to the user.

Authenticity of the certificate: The user must not be able to create their own certificates; this essentially implies that the CA's signature must be part of the user's secret key.

Correctness: The resulting user's public-secret key pair must be such that signature generated with this secret key are valid (and unforgeable) under the public key for the same message.

Implicit certificates from lattices. The main body of this paper will describe potential approaches to creating implicit certificate schemes using lattice-based signatures. In both of the DSS we consider, namely Falcon and Dilithium, the secret key essentially consists of two or more polynomials with small

coefficients and the public key of one or more polynomials with coefficients modulo q . Moreover, the signature consists of a bit string and a polynomial with rather small coefficients, but not as small as the secret polynomial's coefficients. We will briefly explain the design principles of Falcon and Dilithium in Section 2. Through our research, we discover the main recurring difficulties with constructing implicit certificates from Falcon and Dilithium, described as follows.

- Both schemes make use of operations that require certain variables to be large or small for efficiency, security, or correctness reasons. These requirements contradict the requirements of implicit certificates, by yielding signatures of about the same size as the secret key.
- Dilithium's and Falcon's security are based on the learning with errors (LWE) or NTRU problem, respectively. This brings additional constraints in how we can engineer the keys – it is not as simple as adding or subtracting points on an elliptic curve.
- In implicit certificates, both the user and CA are required to share secret information between them, but without leaking their entire secret to each other. Within lattice based cryptography, is a challenge to do so whilst maintaining security and efficiency as it involves storage or computation on polynomials or large matrices, in contrast to points on a curve as in ECC.
- It is challenging to include sufficient information in the reconstruction value to enable reconstruction of the user's key without revealing the CA's secret key or leading to implicit certificates with size larger than that of corresponding explicit certificates. Again, this is due to the delicate requirements on the size and structure of secrets in each of the lattice schemes.

These difficulties lead to many different constructions based on Falcon or Dilithium that failed to achieve all construction principles for implicit certificates. In what follows we first give background information necessary to understand our constructions, before we explain the constructions and the failure reasons.

2. PRELIMINARIES

Notation. We use the following notation throughout the paper. We will generally denote the user by U , the CA by C , and the third-party verifier by V . Moreover, we assign the user a secret/signing key sk_U and a public/verification key pk_U . Similarly we assign sk_C and pk_C to the CA, in addition to their encryption and decryption keys ek_C and dk_C , respectively. We denote the user's certificate issued by a CA as $cert_U$, eventually including U 's reconstruction value R_U . We write σ_m to denote a signature on the message m . If clear from the context, we will omit m in σ_m .

¹ To ensure security, in addition, an appropriate security model should be defined, and the construction's security should be reduced from the security or hardness of another algorithm or mathematical problem.

In addition, we utilize common mathematical notation such as the ring of integers mod q , denoted by $\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z}$, and polynomial rings over $\mathbb{Z}_q$ for some d and q , denoted by $\mathcal{R}_q^+ := \mathbb{Z}_q[x]/\langle x^d + 1 \rangle$. In addition, we call **Enc** and **Dec** the encoding and decoding, respectively, of the reconstruction value and the identity U into/from certificates. We denote the sampling of an element x from a uniform distribution over a finite set S as $x \leftarrow \$S$, the discrete Gaussian distribution over the integers $\mathbb{Z}$ with center c and standard deviation α as $\mathcal{D}_{\mathbb{Z},c,\alpha}$ and the discrete Gaussian distribution over a lattice Λ with center c and standard deviation α as $\mathcal{D}_{\Lambda,c,\alpha}$. When c is not specified, it is implied that $c = 0$. Sampling an element x from $\mathcal{D}_{\Lambda,c,\alpha}$ is denoted by $x \leftarrow \mathcal{D}_{\Lambda,c,\alpha}$. We refer the reader to the Falcon specification [6] for the formal definitions of these functions. Moreover, we denote the Euclidean and the infinity norm of a vector v or polynomial (using the coefficient embedding) as $\|v\|$ and $\|v\|_\infty$.

2.1. Cryptographic primitives

In what follows, we recall the definitions of Digital Signature Schemes (DSS) and certified signature schemes.

Digital signature schemes. A digital signature scheme $\mathcal{S} = (\text{KGen}, \text{Sign}, \text{Vrfy})$ is defined by the following three algorithms:

KGen(): This probabilistic algorithm returns a public-secret key pair (pk, sk) .

Sign (sk, m) : This probabilistic algorithm returns a signature σ upon input message m and secret sk .

Vrfy (pk, σ, m) : This deterministic algorithm returns 1 if the input signature σ is valid for input message m and input public key pk , and 0 otherwise.

We say $\mathcal{S}$ is correct if

$$\Pr[\text{Vrfy}(pk, \text{Sign}(sk, m), m) \mid (pk, sk) \leftarrow \text{KGen}()] = 1.$$

The common security definition for signature schemes is Existential Unforgeability under Chosen Message Attack (EUF-CMA) [26]. It is important to note that there are other important security properties of signatures beyond unforgeability [27–29], such as exclusive ownership [27, 30, 31], message-bound or non-colliding signatures [32], or non-re-signability [28]. We refer the reader to [32], which analyses candidates of the NIST PQ standardization regarding these properties.

(Public-Key) Certificates. Commonly, the public key used to verify the signature of a message is linked to a user's identity using *certificates*. More concretely, RFC 5280 [33, Section 3.1] states that ‘Public key certificates are data structures that bind public key values to subjects. The binding is asserted by having a trusted CA digitally sign each certificate.’ While RFC 5280 defines X.509 certificates which are *explicit* (i.e. the user's public key and the CA's signature over the certificate body are

explicitly included in the certificate), implicit certificates have been standardized as well, such as the (ECQV) scheme [19].

In what follows, we denote with *cbody* the corresponding certificate body which is signed by the CA of an explicit certificate *cert*. For both explicit and implicit certificates, the root certificate *cert_C* includes the CA's public key pk_C and the CA's signature $\sigma_{cert_C} \leftarrow \text{Sign}_C(sk_C, cbody_C)$, i.e. it is self-signed. Every *explicit* cert *cert_U* of a user U consists of the user's public key pk_U , information such as the CA's identity C and the validity period of the certificate, and a signature over U 's public key generated by the CA, denoted by $\sigma_{cert_U} \leftarrow \text{Sign}_C(sk_C, cbody_U)$. Simplified, if a verifier V wants to verify a signature σ_m generated by U on a message m , the verifier first downloads *cert_C* and *cert_U*. V then extracts $pk_U \leftarrow cert_U$ and computes $\text{Vrfy}_C(pk_C, cbody_U, \sigma_{cert_U})$. If Vrfy_C returns 1, the verifier can trust pk_U and use it to verify the signature σ_m , i.e. to compute $\text{Vrfy}_U(pk_U, m, \sigma_m)$.

In contrast, an *implicit* cert *cert_U* of user U consists only of a so-called reconstruction value R_U but *not* of the user's public key or the CA's signature. In order to verify σ_m on m , the verifier V first reconstructs the user's public key pk_U using R_U and pk_C , then calls $\text{Vrfy}_U(pk_U, \sigma_m, m)$, as before. In doing so, authenticity of the user's public key is implied, due to the mechanics of the scheme. Therefore, using implicit certs saves space as long as the size of a signature and the size of the user's public key is larger than the size of the reconstruction value, i.e. $|\sigma_{cert_U}| + |pk_U| > |R_U|$.

In the context of transitioning to a post-quantum secure PKI, hybrid certificates that combine, for example, classical and post-quantum certificates have been proposed [34]. While such hybrid certificates are often discussed as combinations of explicit certificates, they can also be generalized as combinations of implicit and/or explicit certificates, depending on the respective certificate structure.

Certified signature schemes. The combination of signature schemes with certified verification keys (using implicit or explicit certificates) has been formalized by Boldyreva, Fischlin, Palacio and Warinschi [17]. We recall the definition of *certified signature schemes* next.

We define a certified signature scheme $\mathcal{C} = (\text{KGen}, (\text{Gen}_C, \text{Gen}_U), \text{Sign}, \text{Vrfy})$ via the following four algorithms and protocols.

KGen(): This probabilistic algorithm returns a public-secret key pair (pk_C, sk_C) of the CA C .

(Gen_C(U, sk_C), Gen_U(U, pk_C)): This is a probabilistic, interactive public-key registration protocol between two parties, namely the user U and the CA C who execute their probabilistic algorithms **Gen_C** and **Gen_U**, respectively. The algorithm **Gen_C** returns $(U, pk_U, cert_U)$ upon input sk_C and U with pk_U being the user's public key, and *cert_U* is the resulting issued certificate. The algorithm **Gen_U** returns $(U, pk_U, sk_U, cert_U)$ (that is possibly different from **Gen_C**'s output) upon input identity U of a user and

the public key pk_C corresponding to sk_C . Either party can quit the execution at any time, in which case the output of the respective party is $\perp$.

Sign($U, sk_U, cert_U, pk_C, m$): This randomized signing algorithm returns a signature σ upon input identity U , a secret key sk_U , a certificate $cert_U$, the CA's public key pk_C , and a message m .

Vrfy($U, pk_U, cert_U, pk_C, m, \sigma$): This deterministic algorithm returns 1 or 0 upon input identity U , public key pk_U , certificate $cert_U$, public key pk_C , message m , and signature σ . If it returns 1, we call σ a valid signature for m relative to $(U, pk_U, cert_U, pk_C)$. Otherwise, it is called invalid.

In the case $\mathcal{C}$ uses implicit certificates, the verification algorithm **Vrfy** internally reconstructs U 's public key, before verifying the signature. We call $\mathcal{C}$ correct if all m and U satisfy $\text{Vrfy}(U, pk_U, cert_U, pk_C, m, \text{Sign}(U, sk_U, cert_U, pk_C, m)) = 1$, for $((U, pk_U, cert_U), (U, pk_U, sk_U, cert_U)) \leftarrow (\text{Gen}_C(sk_C), \text{Gen}_U(U, pk_C))$ and $(pk_C, sk_C) \leftarrow \text{KGen}()$. See [17, Definition 4.1] for a definition of unforgeability of a certified signature scheme.

In all our constructions, we assume that all users and verifiers know the CA's public key.

A note on identity-based signature schemes. On the surface, implicit certificate schemes resemble identity-based signatures (IBS). However, the difference is as follows. In IBS, a Trusted Third Party (TTP) derives sk_U from the user's identity U by use of a master secret key, and communicates sk_U to the user via a secure channel. A verifier does not need to obtain the user/signer's certificate, as all they need is the signer's identity. On the contrary, as outlined in the construction principles, implicit certificate schemes assume only the user knows their secret key sk_U , and the extent of trust in the TTP, in this case the (CA), only goes as far as being assured that they will bind the user's identity with the intended public key pk_U .

The threat model of the two concepts is somewhat similar: the key escrow problem of IBS, that is, the fact that the TTP can sign any message on behalf of the user, can be replicated in the implicit certificate setting by the CA creating a fraudulent certificate for the user with a public key for which only the CA knows the corresponding secret key.

Although the two concepts of IBS and implicit certificates share similarities, they are not directly interchangeable. For example, IBS would not be sufficient for V2V in which messages and signatures are broadcast and in which the receiving vehicle usually does not know the signer's identity. Specifically, the IEEE 1609.2 standard[21] employs pseudonyms to support the anonymity of the vehicle. If using IBS, the sending vehicle would be required to broadcast their ID alongside with their message to claim their authenticity, or alternatively have to re-derive a new private key each time they refreshed their pseudonym. This would constitute further effort and adjustments to the standard, which is out of scope of this paper. For

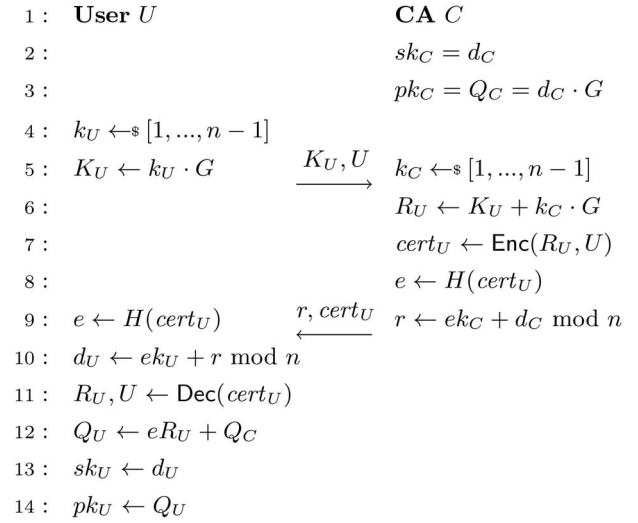


FIGURE 1. ECQV implicit cert scheme [19] with G being the group generator.

a more detailed discussion on IBS and its relation to PKI, see [35].

2.2. Description of the ECQV scheme

Let G be the elliptic curve base point that generates a subgroup of large prime order n and let q be the field size. We depict the (ECQV) scheme in Figure 1. Note that the variables denoted by n, e and r are unrelated to those featured in the lattice schemes used later on. The public encoding and decoding functions are denoted by $\text{Enc}()$ and $\text{Dec}()$ respectively. The idea of the ECQV scheme by Qu and Vanstone [19] is as follows. The user chooses a kind of ephemeral key, $K_U \leftarrow k_U \cdot G$, and sends it to the CA. The CA chooses a randomness, k_C , that is added to the user's ephemeral key, becoming the reconstruction value R_U that is included in the issued certificate along with the user's identity U , via the encoding function. In addition, the randomness k_C is also used to compute a value r that is inspired by ECDSA signatures. It is important to note that the value r is, however, not included in the issued certificate but sent to the user (over an unprotected channel). Upon receipt of the certificate, any individual, which could be the user themselves, or a verifier wishing to verify a signature of the user, can decode it to retrieve the reconstruction value R_U and, using the CA's public key Q_C and the public hash output of $cert_U$, derive the user's public key. The final user secret key is then generated from the secret part of the ephemeral key k_U which is added to the value r (the CA's 'signature' on the user's certificate). This is possible because essentially all elements in this scheme are integers or points on the elliptic curve and hence, can easily be multiplied or added.

The authors of [18] show that the composition of ECQV with ECDSA is secure against passive adversaries, in the random oracle and generic group model. There has been no analysis against active adversaries.

2.3. Hard problems over lattices

The implicit certificate schemes proposed in this research are based on lattices, one of the candidate families for post-quantum secure cryptography. We now introduce the hard problems over lattices upon which the security of our constructions depend.

Let $k, \ell, d, n \in \mathbb{Z}$. A lattice is a discrete additive subgroup of $\mathbb{R}^{kd}$. Let $B = \{b_1, b_2, \dots, b_{\ell d}\} \subseteq \mathbb{R}^{kd}$ be a set of linearly independent vectors. We define the lattice Λ with basis B as

$$\Lambda(B) = \left\{ \sum_{i=1}^{\ell d} b_i x_i : x_i \in \mathbb{Z}^{kd} \right\}.$$

In (Mod)Falcon, the lattices are full-rank, so $k = \ell$.

We start with the LWE problem which is defined as follows. Given a uniform matrix $\mathbf{A} \in \mathbb{Z}_q^{kd \times \ell d}$ and a vector $b = \mathbf{A} \cdot s + e \in \mathbb{Z}_q^{kd}$ with s and e being vectors with small coefficients, recover the vector $s \in \mathbb{Z}^{kd}$. The short vectors $s, e \in \mathbb{Z}^{kd}$ are often sampled from a discrete Gaussian distribution but other distributions are possible as for Dilithium, which stipulates a uniform distribution over a small interval $S_\eta = [-\eta, \eta]$, where η is a small integer. The Ring Learning With Errors (R-LWE) and Module Learning With Errors (M-LWE) problems refer to LWE over ideal and module lattices with elements in $\mathcal{R}_q^+$ and $(\mathcal{R}_q^+)^n$, respectively.

The Falcon signature scheme relies on a variant of the NTRU problem, stated as follows, with parameter n fixed as $n = 1$. Let $g \in (\mathcal{R}_q^+)^n$, $\mathbf{F} \in (\mathcal{R}_q^+)^{n \times n}$, sampled according to some specified distribution. Throughout this paper, this distribution will be $\mathcal{D}_\alpha$, with standard deviation $\alpha = \alpha_g = 1.17\sqrt{\frac{q}{2d}}$ for g and $\alpha = \alpha_{\mathbf{F}} = 1.17\sqrt{\frac{q}{(n+1)d}}$ for $\mathbf{F}$. When $n = 1$, we denote $\mathbf{F}$ by f to make it clear that it is a polynomial, and in line with the notation of [6]. Given $h^t = \mathbf{F}^{-1} \cdot g^t \in (\mathcal{R}_q^+)^n$, recover $\mathbf{F}$ and g . The ModFalcon scheme [36] extends this problem to lattices with $n \geq 2$, so g and h are extended from single polynomials to become vectors of polynomials, and $\mathbf{F}$ from a vector of polynomials to a matrix of polynomials, where all polynomials are of degree d .

Algorithm 1 KGen_{Falcon}

Require: -

Ensure: (sk, pk)

- 1: $g, f \leftarrow \mathcal{D}_\sigma(\mathbb{Z}^d)$, f being invertible
 - 2: $h \leftarrow gf^{-1} \bmod q$
 - 3: $G, F \leftarrow \text{NTRUSolve}(g, f)$
 - 4: $pk \leftarrow h$
 - 5: $sk \leftarrow (g, -f, G, F)$
 - 6: **return** (sk, pk)
-

Algorithm 2 Sign_{Falcon}

Require: $sk = (g, -f, G, F)$, m

Ensure: $\sigma = (r, z_2)$

- 1: $r \leftarrow \$_$
 - 2: $c \leftarrow H(r||m)$
 - 3: $(z_1, z_2) \leftarrow \text{FS}(c, sk)$ with $z_1 + z_2 h = c \bmod q$
 - 4: **return** $\sigma = (r, z_2)$
-

Algorithm 3 Vrfy_{Falcon}

Require: $pk = h, m, \sigma = (r, z_2)$

Ensure: $\{0, 1\}$

- 1: $c \leftarrow H(r||m)$
 - 2: $z_1 \leftarrow c - z_2 h \bmod q$
 - 3: **if** $||(z_1, z_2)|| \leq \beta$ **then return** 1
 - 4: **else return** 0
-

2.4. Description of Falcon and Dilithium

Falcon and Dilithium are the only lattice-based signature finalists selected for standardization by NIST so far [8], and hence in this paper, we tried to construct implicit certificates from them.

The core idea of the Falcon [6] signature scheme key generation is to build two lattice bases (one private, one public) for the same lattice consisting of 2×2 cyclic blocks. The signature (z_1, z_2) is a short vector in this lattice, which can only be found with the knowledge of the private basis and can be verified as being in the lattice by knowledge of the public basis. The private key is comprised of small polynomials g, f, G , and F that define the cyclic blocks. In Algorithm 1, g, f are sampled from a discrete Gaussian distribution which become the input to a sub-algorithm **NTRUSolve** to obtain a valid G, F such that $f \cdot G - g \cdot F = q$. We often omit G, F when referring to a Falcon secret key pair in this paper as it does not impact our design decisions. The public key is a large polynomial $h = g \cdot f^{-1}$, which defines the same lattice via a different cyclic block matrix. At the core of signature generation, Algorithm 2, lies Falcon's sampling function **FS** : $(\mathcal{R}_q^+)^{(n+2)d} \rightarrow \mathbb{R}_q^+, (c, sk) \mapsto (z_1, z_2)$ which takes as input the secret key, i.e. the small polynomials, and a target value c , and samples values (z_1, z_2) with discrete Gaussian distribution around c such that $z_1 + z_2 h = c \bmod q$. It is important to note that $||(z_1, z_2)||$ is small, more precisely, $||(z_1, z_2)|| \leq \beta$, for some parameter β . As only z_2 and the randomness r are part of the signature, verification (see Algorithm 3) first computes z_2 before it checks the size of $||(z_1, z_2)||$. The last step essentially means to check that (z_1, z_2) are close to c which can only be the case if the secret has been known when using the sampler **FS**. By H we denote the hash function used in Falcon, specifically SHAKE-256, which maps an input string to a polynomial in $\mathcal{R}_q^+$. The security of Falcon is based on a variant of the NTRU problem.

Algorithm 4 KGen_{Dilithium}**Require:** -**Ensure:** (sk, pk)

```

1:  $\rho \leftarrow \{0, 1\}^{256}$ 
2:  $A \in \mathcal{R}_q^{k \times \ell} \leftarrow \text{Expand}(\rho)$ 
3:  $(s, e) \leftarrow S_\eta^l \times S_\eta^k$ 
4:  $t \leftarrow As + e \pmod q$ 
5:  $sk \leftarrow (\rho, t, s, e), pk \leftarrow (\rho, t)$ 
6: return  $(sk, pk)$ 

```

Algorithm 5 Sign_{Dilithium}**Require:** $sk = (\rho, t, s, e), m$ **Ensure:** $\sigma = (z, c)$

```

1:  $A \leftarrow \text{Expand}(\rho)$ 
2: while  $z \leftarrow \perp$  do
3:    $y \leftarrow S_{\gamma_1-1}^l$ 
4:    $c \leftarrow H(\text{HighBits}(Ay) || m)$ 
5:    $z \leftarrow y + cs$ 
6:   if  $\|z\|_\infty \geq \gamma_1 - \beta$  or  $\|\text{LowBits}(Ay - ce)\|_\infty \geq \gamma_2 - \beta$  then  $z \leftarrow \perp$ 
7: return  $\sigma = (z, c)$ 

```

Algorithm 6 Vrfy_{Dilithium}**Require:** $pk, m, \sigma = (z, c)$ **Ensure:** $\{0, 1\}$

```

1:  $A \leftarrow \text{Expand}(\rho)$ 
2: if  $c == H(\text{HighBits}(Az - ct) || m)$  and  $\|z\|_\infty \leq \gamma_1 - \beta$  then return 1
3: else return 0

```

The arguably biggest advantage is Falcon's small signature and public key size. However the implementation of the Falcon scheme is more complicated than that of Dilithium. At the forefront, the Gaussian sampling procedures involve complex numbers, which require Floating Point Arithmetic (FPA). For best performance, dedicated processors for FPA are recommended, otherwise the Falcon signing procedure is four times slower than Dilithium [37]. In addition, Falcon only comes with two parameter sets; a third one was discarded as it required a ternary polynomial ring and therefore made the implementation more complex. The strict conditions on and interdependence of the parameters means it is not possible to generate an set corresponding to an intermediate security level over the ring $\mathcal{R}_q^+$.

Chuengsatiansup, Prest, Stehlé, Wallet, and Xagawa introduced a generalized variant of Falcon over modules, called ModFalcon [36]. While in the original Falcon $n = 1$, ModFalcon generalizes this to cases where $n \geq 2$, as explained in section 2.3, and depends on the Mod-NTRU problem for security. We made the choice to use ModFalcon in one of our constructions that combines ModFalcon with Dilithium. However, we will only present the Falcon pseudocode here as it highlights the main features of both schemes.

Compared to Falcon, Dilithium [7] comes with a larger signature and public key size but with a considerably easier

to implement design and less restrictive requirements on the choice of parameters. Dilithium is a Fiat-Shamir signature scheme [38] over modules, and as such the design idea can be described as follows. In Algorithm 4, we can see the secret key consists of two vectors s, e of polynomials with small coefficients. The public key is a corresponding M-LWE sample, i.e. $(A, t = As + e \pmod q)$. During signature generation (see Algorithm 5, where H denotes Dilithium's hash function which maps an string to a polynomial in $\mathcal{R}_q^+$), first a randomness y is sampled which is the $-$ with A multiplied and $-$ hashed together with the message to compute c . Afterwards, $z = y + cs$ is computed, where intuitively the randomness 'hides' the secret s . Hence, the randomness has to be larger than the secret. This means that the resulting signature is also larger than the secret key, which will be an important consideration when constructing implicit certificates. However, the signature (z, c) is only returned if it is small enough, otherwise a new randomness is sampled. This is often referred as the rejection sampling of lattice-based signatures and is needed to ensure that the distribution of signatures is independent of the secret key. The security of Dilithium depends on the M-LWE and Self-Target Short Integer Solution (SelfTargetSIS) problems, where we refer to [7] for a definition of the latter as we do not rely on this problem directly in our presented constructions.

Kyber [39] is a Public Key Encryption (PKE)/KEM, due to be standardized by NIST [8]. We refrain from going into specifics here as it is not critical to our constructions, but we recommend it for use in Section 4. We display parameters for the aforementioned schemes for NIST's security Level 2 (equivalent to SHA256 collision search, i.e. 128-bit security) that are relevant for our constructions in Table 1. We additionally include the higher security Dilithium sets which we will require. The important takeaway from this table is that by computing the product of the column d and n (or d and $k = \ell$ for Dilithium) as appropriate, we obtain the dimension of the corresponding $\mathbb{Z}$ -lattice. We require these dimensions to align when combining two schemes. As can be seen, this may require tweaking the recommended parameters, such as the modulus, and so we have to be mindful of the effect on security. We discuss this further under the relevant construction.

3. EMULATION OF ECQV

In this section we present our (failed) approaches to mirror the ECQV scheme using Falcon, Dilithium and a combination of both of the schemes.

3.1. Emulating ECQV using Falcon

In what follows, we describe different ideas of how to construct implicit certificates using Falcon and the reasons why the constructions fail. We discover the obstacles for failing to mirror ECQV with Falcon to be the very small Falcon secret keys (Section 3.1.1), that the input polynomials to the Falcon sampling function must be small (Section 3.1), and the restrictive structure of the underlying NTRU problem (Section 5).

TABLE 1. Instantiations of Falcon, ModFalcon, Dilithium and Kyber for NIST security level 2. The parameters η and $\gamma_1 - \beta$ are the maximum coefficient sizes of the secret key and signatures, respectively.

Scheme	d	n	(k, ℓ)	q	η	$\gamma_1 - \beta$
Falcon-512	512	1	-	12,289	31	2047
ModFalcon-512	512	2	-	12,289	31	2047
Dilithium-II	256	-	4,4	8,380,417	2	$2^{17} - 78$
Dilithium-III	256	-	6,7	8,380,417	4	$2^{19} - 196$
Dilithium-V	256	-	8,7	8,380,417	2	$2^{19} - 120$
Kyber-512	256	-	2	3,329	3	-

In all three constructions, we denote the Falcon signature scheme as $\mathcal{S} = (\text{KGen}, \text{Sign}, \text{Vrfy})$ and we assume the CA C has generated a Falcon key pair $sk_C = (g_C, f_C)$ and $pk_C = h_C = g_C \cdot f_C^{-1}$.

In ECQV, the value k_C can either be viewed as randomness or as an ephemeral secret key of C . Due to the structure of the NTRU problem, there is no similar keying material to satisfy this double-use in the Falcon signature scheme. Therefore, in the first two of our constructions, we model each of the uses individually.

3.1.1. Mirroring k_C as randomness

Crypto primitives	Signature schemes
Instantiations	Falcon
Failure reason	No authenticity and correctness

In this very first construction (see Figure 2), the user U first generates an ephemeral Falcon key pair with $sk_U \leftarrow (g_U, f_U)$ and $pk_U \leftarrow h_U = g_U \cdot f_U^{-1}$. The CA C starts by generating a randomness r_U (similar to k_C in ECQV, Figure 1) in the form of a ring polynomial, then computes the reconstruction value $R_U \leftarrow h_U \cdot r_U$ (analogous to $R_U \leftarrow K_U + k_C G$ in ECQV). The CA chooses also the randomness for signing as per the Falcon signing algorithm, in the form of a bit string r_C , and encodes it together with R_U and the user identity U in the certificate $cert_U$. Afterwards, the CA essentially signs the certificate, i.e. hashes r_C together with the certificate to get c and uses Falcon's trapdoor function to generate $(z_1, z_2) \leftarrow \text{FS}(r_U^{-1} c, sk_C)$ such that $z_1 + z_2 \cdot h = r_U^{-1} \cdot c$. The user can compute r_U and z_1 and sets the secret key and public key as in Figure 2.

A verifier can compute U 's public key as $R_U \cdot H(r_C || cert_U)$ since the value R_U and r_C are both included in $cert_U$ and

$$\begin{aligned} R_U \cdot H(r_C || cert_U) &= h_U \cdot r_U \cdot c \\ &= g_{\text{new}} \cdot f_U^{-1} \cdot (z_1 + z_2 \cdot h_C). \end{aligned}$$

In this sense, the scheme is correct.

1 : User U	CA C
2 : $((g_U, f_U), h_U) \leftarrow \text{KGen}() \xrightarrow{h_U, U}$	$r_U \leftarrow \mathcal{R}_q$
3 :	$r_C \leftarrow \mathcal{R}_{\{0,1\}^{128}}$
4 :	$R_U \leftarrow h_U \cdot r_U$
5 :	$cert_U \leftarrow \text{Enc}(R_U, r_C, U)$
6 :	$c \leftarrow H(r_C cert_U)$
7 :	$(z_1, z_2) \leftarrow \text{FS}(c, sk_C)$
8 : $(z_2, r_C) \leftarrow \sigma$	$\sigma, cert_U \xleftarrow{\quad} \sigma \leftarrow (z_2, r_C)$
9 : $c \leftarrow H(r_C cert_U)$	
10 : $r_U \leftarrow R_U \cdot h_U^{-1}$	
11 : $z_1 \leftarrow c - z_2 \cdot h_C$	
12 : $g_{\text{new}} \leftarrow g_U r_U (z_1 + z_2 h_C)$	
13 : $sk_U \leftarrow (g_{\text{new}}, f_U)$	
14 : $pk_U \leftarrow R_U \cdot c$	

FIGURE 2. Emulating ECQV using Falcon, mirroring k_C of ECQV as randomness r_U .

However, there are several issues with this naive approach. The most severe is that authenticity of the certificate $cert_U$ is not guaranteed as the user can generate their own certificates. The reason is that the CA's public key is not needed in the verifier's computation of U 's public key, and similarly, neither the CA's secret key nor signature are required in the user secret key computation. Hence, a user can generate its own signature without the need of the CA's secret key. This stems from the fact that line 11 is somewhat redundant, but we included this step to demonstrate the mirroring of the ECQV scheme. At the source of this is that Figure 2 actually does not mirror ECQV exactly, in particular the signing of the certificate by C is simplified and does not involve r_U as it does in ECQV. A potential change could be to give r_U somehow as input to the Falcon sampler, e.g., input $r_U \cdot c$ instead of c . However, we did not find a way to communicate r_U to the verifier such that they can compute U 's corresponding public key such that 1) C 's secret is not leaked and 2) the size of the resulting $cert_U$ is smaller than a corresponding explicit certificate.

Moreover, the resulting secret key of U is much larger than in the original Falcon design. The Falcon key generation is very particular about the size of the secret keys. They are sampled

1 : User U 2 : $((g_U, f_U), h_U) \leftarrow \text{KGen}()$ 3 : 4 : 5 : 6 : 7 : 8 : $r \leftarrow \text{Enc}_1(r')$ 9 : $c \leftarrow H(rh_U h_C \text{cert}_U)$ 10 : $sk_U \leftarrow c \cdot g_U, f_U$ 11 : $pk_U \leftarrow c \cdot h_U$	CA C $r' \leftarrow_{\$} \{0, 1\}^{128}$ $r \leftarrow \text{Enc}_1(r')$ $R_U \leftarrow (r', h_U)$ $\text{cert}_U \leftarrow \text{Enc}(r', h_U, U)$ $c \leftarrow H(rh_U h_C \text{cert}_U)$ $(z_1, z_2) \leftarrow \text{FS}(c, sk_C)$ $\sigma \leftarrow (z_2, r)$
---	---

FIGURE 3. Emulating ECQV using Falcon by including h_C in pk_U and mirroring k_C as an NTRU instance.

from a Gaussian distribution with standard deviation α set so that the expected norm $\|f, g\|$ of the output keys is $1.17\sqrt{q} = 1.17\sqrt{12,289} \approx 129.7$. This ensures that when they are used in the sampler, the square norm of the resulting signature is $\leq \beta^2 = 34\,034\,726$ for Falcon-512. Shorter signatures imply higher security, but noise is added to prevent them leaking the secret basis. So, it can be expected that signatures generated with this construction's large secret key are not valid or not secure as the Falcon sampler is not designed for such large keys.

Another variant of the same idea (and the same issues) is given in Figure 3. This variant shows that it is not sufficient for the CA's public key to be needed in the computation of pk_U but intertwining the CA's signature in the user's secret key is additionally required. In this variant, the problem is again that the utilized information is publicly available and so the user is able to generate their own valid certificate.

In Figure 3, we use (in addition to the certificate encoding function Enc) the function $\text{Enc}_1^2 : \{0, 1\}^{320} \rightarrow \mathcal{R}_q^+$, which maps a bit string r' to a polynomial r in the polynomial ring $\mathcal{R}_q^+$. If instantiating Falcon as the signature scheme, this function is not necessary as this is taken care of within the signing algorithm.

3.1.2. Mirroring k_C as additional NTRU instance

<i>Crypto primitives</i>	Signature schemes
<i>Instantiations</i>	Falcon
<i>Failure reason</i>	Correctness

In our first construction, we mirrored k_C as a random polynomial. Next, we show that unfortunately also the second possibility—mirroring k_C as an additional NTRU instance—does not lead to an efficient and secure implicit certificate scheme. This supports our observation that the core of ECQV lies in the versatility of elements used.

1 : User U 2 : $((g_U, f_U), h_U) \leftarrow \text{KGen}()$ 3 : 4 : 5 : 6 : 7 : 8 : 9 : $R_U \leftarrow \text{cert}_U$ 10 : $h'_C \leftarrow R_U \cdot h_U^{-1}$ 11 : $c \leftarrow H(r \text{cert}_U)$ 12 : $z_1 \leftarrow c - z_2 h_C h'_C$ 13 : $g_{\text{new}} \leftarrow g_U \cdot (c - z_1) \cdot z_2^{-1}$ 14 : $sk_U \leftarrow (g_{\text{new}}, f_U)$ 15 : $pk_U \leftarrow R_U \cdot h_C$	CA C $h_U, U \xrightarrow{\quad} R_U \leftarrow h_U \cdot h'_C$ $\text{cert}_U \leftarrow \text{Enc}(R_U, U)$ $r \leftarrow_{\$} \{0, 1\}^{128}$ $c \leftarrow H(r \text{cert}_U)$ $g \leftarrow g'_C g_C, f \leftarrow f'_C f_C$ $(z_1, z_2) \leftarrow \text{FS}(c, g, f)$ $\sigma \leftarrow (r, z_2)$
--	--

FIGURE 4. Emulating ECQV using Falcon, mirroring k_C of ECQV as additional key (sk'_C, pk'_C).

In contrast to the first construction in Section 3.1.1, we change the input of the Falcon sampler to consider both sk_C and sk'_C . More concretely, the CA computes $(z_1, z_2) \leftarrow \text{FS}(c, g'_C g_C, f'_C f_C)$ such that $z_1 + z_2 h_C h'_C = c$. The user can compute R_U, h'_C, c , and z_1 as depicted in Figure 4. The user's secret key is then defined as $sk_U \leftarrow (g_U \cdot (c - z_1) \cdot z_2^{-1}, f_U)$, while the public key can be computed as $R_U \cdot h_C$ by anyone. It holds that

$$\begin{aligned}
pk_U &= R_U \cdot h_C = h_U \cdot h'_C \cdot h_C \\
&= h_U \cdot (c - z_1) \cdot z_2^{-1} = g_U \cdot (c - z_1) \cdot z_2^{-1} \cdot f_U^{-1}.
\end{aligned}$$

It is important to point out that in the original Falcon specification [6], z_2 is not necessarily invertible. However, in Section 3.12 of the Falcon specifications, a key recovery mode is proposed that also requires z_2 to be invertible. We omit explaining the details of our construction but depict them in Figure 4.

The problem with this approach is that the resulting secret key is again too large to satisfy the requirement on the Falcon keys, and therefore would be unsuitable for using as the trapdoor sampler input.

3.2. Emulating ECQV using Dilithium

Using Dilithium to mirror the operations in ECQV straightforwardly is not possible due to conflicting requirements on the secret key and signature sizes as explained next.

To construct implicit certificates similarly to ECQV, it is necessary to include the CA's signature in the user's secret key to ensure authenticity of the certificate. As shown in Algorithm 5, Dilithium's signature consists of the element $z \leftarrow y + cs$ where y is the randomness, c is a hash value, and s is part of the secret. Intuitively, the randomness *hides* the secret. To this end,

² The exponent 320 is following from the Falcon specifications.

the coefficients of the randomness (and hence the signature) must be larger than the secret coefficients. To keep the signature size small, generally one optimization goal is to keep secret and randomness as small as possible. Hence, constructing a variant of Dilithium from LWE with uniformly sampled secret over $\mathcal{R}_q^+$ instead of $\mathcal{S}_\eta$ will be less efficient overall as user's signatures would become much larger.

One approach to overcome this difficulty could be to use different Dilithium instantiations for the CA and the user. For example, the CA could use Dilithium-II which provides a security of 128 bit. However, the coefficients of the signature of this instance are at most $\gamma_1 - \beta = 2^{17} - 78$ in absolute value, which would still not fit into the secret key of Dilithium-III, which allows for secret key coefficients of at most $\eta = 3$. Evidently, if Dilithium did have a larger η , this would result in very large public key and signature sizes relative to the provided bit security, as the masking polynomial y would require even larger coefficients to hide the secret.

Other obstacles when constructing implicit certificates from Dilithium could be the structure of the M-LWE tuples. More concretely, the error term in $b = \mathbf{A}s + e$ might obstruct homomorphic operations. In addition, the error term can not be reused for different secret terms with the same $\mathbf{A}$, otherwise the corresponding M-LWE instances are easy to solve. As an example of a construction that failed because of this, see Section 3.3.

As a consequence, we do not present a concrete construction that uses solely Dilithium. However, we do make use of Dilithium in combination with Falcon, in our next construction.

3.3. Combining (Mod)Falcon and Dilithium

<i>Crypto primitives</i>	Two DSS
<i>Instantiations</i>	Falcon, Dilithium
<i>Failure reason</i>	Correctness, signature size

Our most promising construction, depicted in Figure 5, originated in the combination of the two signature schemes (Mod)Falcon and Dilithium. The idea behind this construction is that by using Falcon during certificate generation, and Dilithium as the user's signature scheme, we can leverage the fact that Falcon has smaller signatures (666 bytes), but Dilithium allows for larger secret keys (2 528 bytes compared to Falcon's 1 281 bytes for Level 2 security), to incorporate the CA's signature into the user's secret key, as per the penultimate construction principle listed in Section 1. This comes at the cost of additional security assumptions, namely the hardness of NTRU and LWE and SelfTargetSIS.

However, depending on the parameter sets we chose, it may be more suitable to use the generalization of Falcon over module lattices, ModFalcon, as introduced in Section 2.

In our description, we denote Dilithium as $\mathcal{S}_D = (\text{KGen}_D, \text{Sign}_D, \text{Vrfy}_D)$ and Falcon as $\mathcal{S}_F = (\text{KGen}_F, \text{Sign}_F, \text{Vrfy}_F)$.

1 : User U	CA C
2 :	$((g_C, f_C), h_C) \leftarrow \text{KGen}_F()$
3 :	$v_C^{-1} \leftarrow_{\sigma} \mathcal{R}_q^\ell$
4 :	$f_C'^{-1} \leftarrow v_C^{-1} \cdot h_C$
5 :	$sk_C \leftarrow (g_C, f_C)$
6 :	$pk_C \leftarrow (h_C, f_C')$
7 :	
8 : $(\rho, t, s_U, e_U) \leftarrow \text{KGen}_D()$	
9 :	$\xrightarrow{\rho, t, U} r \leftarrow_{\$} \{0, 1\}^{128}$
10 :	$R_U \leftarrow (\rho, t, r)$
11 :	$cert_U \leftarrow \text{Enc}(R_U, U)$
12 :	$c \leftarrow (\text{H}(r cert_U), 0, \dots, 0)$
13 :	$(z_1, z_2) \leftarrow \text{FS}(c, g_C f_C', f_C)$
14 : $(\rho, t, r, U) \leftarrow cert_U$	$\xleftarrow{z_2, cert_U}$
15 : $A \leftarrow \text{Expand}(\rho)$	
16 : $c \leftarrow \text{H}(r cert_U)$	
17 : $v_C^{-1} \leftarrow h_C^{-1} \cdot f_C'^{-1}$	
18 : $z_1 \leftarrow c - z_2 \cdot h_C \cdot f_C'$	
19 : $s_{\text{new}} \leftarrow s_U + z_1 v_C^{-1} + z_2$	
20 : $sk_U \leftarrow (s_{\text{new}}, e_U)$	
21 : $pk_U \leftarrow t + A \cdot c \cdot h_C^{-1} f_C'^{-1}$	

FIGURE 5. Emulating ECQV from (Mod)Falcon and Dilithium, where $\mathcal{S}_\eta$ is the subset of $\mathcal{R}_q$ with polynomial coefficients $\leq \eta$.

Moreover, we define $d_F \cdot n = d_D \cdot \ell$, where d is the polynomial degree in each scheme, n is ModFalcon's ring dimension and ℓ is the dimension of the public key matrix as denoted in Dilithium. This only works for Dilithium-II, where $k = \ell$, as all the (Mod)Falcon parameter sets are full-rank. If we wanted a different security level, we would have to derive new parameter sets outside the specification.

In the construction given in Figure 5, we can see that the CA generates a Falcon key pair, alongside a sampled polynomial v_C^{-1} . This additional value allows a decrease in the size of the signature values when added to the user's secret key. More concretely, this enables us to add $z_1 \cdot v_C^{-1} + z_2$ instead of $z_1 + z_2 \cdot h_C$ with h_C having much larger coefficients than v_C^{-1} .

The user generates a pair of Dilithium keys as per Algorithm 4. The public key pk_U can be reconstructed from the reconstruction value R_U in the certificate $cert_U$ and the CA's public key $pk_C = (h_C, f_C')$ by U and any verifier V . The secret key sk_U can be computed only by U since s_U and e_U are only known by U (under the LWE assumption). Moreover, the following equations hold, which implies the user key is an LWE tuple:

$$\begin{aligned}
 pk_U &= t + A \cdot c \cdot h_C^{-1} \cdot f_C'^{-1} \\
 &= t + A \cdot c \cdot v_C^{-1} \\
 &= As_U + e_U + A(z_2 v^{-1} + z_1) \\
 &= As_U + e_U + A(z_1 v^{-1} + z_2) \\
 &= A \cdot (s_U + z_1 v_C^{-1} + z_2) + e_U.
 \end{aligned}$$

There are three main issues with this approach: issues with the correctness of the CA and the user's signatures, resulting bandwidth usage, and a security concern.

Correctness issues. The issue is that the size of the signature coefficients of Falcon are still much larger (up to 2047 compared to 2, see Table 1) than the allowed secret key size of Dilithium, and hence, the signature over $cert_U$ cannot be added to the secret key. Another issue arises from the construction of Falcon signatures as explained next. Originally, $(z_1, z_2) \leftarrow \text{FS}(c, g_C, f_C)$ such that $z_1 + z_2 h_C = c$. This means that while (z_1, z_2) consists of rather small polynomials, to reconstruct pk_U using pk_C , we need to intertwine the user's public key with h_C . As briefly mentioned above, to overcome this issue we introduced the small element v_C^{-1} , used throughout signature generation, to decrease the size of the element that is then added to the secret key. Unfortunately, to not reveal the CA's secret key and by construction of Falcon, we cannot decrease the size of the element to be added to the secret key much further and it is still too large to be added to Dilithium parameters.

Let's instantiate the scheme with the ModFalcon512 (in key recovery (KR) mode, according to [6, Section 3.12]) and Dilithium-II parameters given in Table 1. This works as $d_F \cdot n$ from ModFalcon512 is equal to $k \cdot d_D = \ell \cdot d_D$ of Dilithium-II. However the moduli q are very different in each scheme. Disregarding this for now, we can see that if we did instantiate this scheme with these values of d, n, k, ℓ , then (z_1, z_2) would be of size 1438 bytes. This means sk_U would have a size of $|s_U| + |(z_1, z_2)| = 2528 + 1438 = 3966$ bytes. Furthermore, we would need to increase the maximum coefficient size η of the masking polynomial y from $\eta = 2$ to 2047, as can be seen in Table 1.

Bandwidth. Another consideration is whether we are still reaping the benefits of implicit certificates with this design. Recall from Section 1 that the main motivation for utilising them is to reduce the required bandwidth in sending an authenticated message. However, if we compare our hybrid design with an explicit certificate scheme instantiating Falcon as both the CA and user's signature scheme, we find that the size of an explicit certificate and user signature is approximately $|pk_U| + |\sigma_{cert_U}| + |\sigma_m| = 2229$ bytes. Yet a Dilithium signature alone, as transmitted by the user in this design, is already 2420 bytes. Hence this defeats the purpose of using implicit certificates.

Security concern. Our construction is not secure due to the structure of the LWE problem. More concretely, we use the error term e_U twice which makes it easier to solve the resulting LWE instance, namely we use it in t and pk_U . To overcome this issue is challenging. One approach is to let C choose another error term e'_U that is added to t and saved as R_U , i.e., instead of $t = \mathbf{A}s_U + e_U$, $t = \mathbf{A}s_U + (e_U + e'_U)$ is saved in R_U

and used in pk_U . The resulting user keys would then be $sk_U = (s_U + z_1 v_C^{-1} + z_2, e_U + e'_U)$ and $pk_U \leftarrow t + \mathbf{A} \cdot c \cdot h_C^{-1} \cdot f_C'^{-1} = \mathbf{A} \cdot (s_U + z_1 v_C^{-1} + z_2) + e_U + e'_U$. Having access to $\mathbf{A}s_U + e_U$ and $t = \mathbf{A}s_U + (e_U + e'_U)$, the user (and only the user) should be able to compute the entire secret. This comes, however, at the cost of the CA being trusted and honestly adding the extra error term. We discuss this additional assumption in more detail in Section 4.

4. CONSTRUCTION USING ENCRYPTION

<i>Crypto primitives</i>	DSS, PKE
<i>Instantiations</i>	Dilithium, Kyber
<i>Failure reason</i>	No authenticity of $cert_U$
<i>Additional Assumptions</i>	Trusted CA, additional keys

This construction assumes a trusted CA by which we mean that the CA knows the user's *entire* secret key and violates the third construction principle listed in Section 1. This can be seen as a higher level of trust than simply signing public keys. It is common to trust the CA not to sign an invalid public key for which it has the corresponding private key which would allow it to authenticate messages on behalf of the user. However, in the case that the CA knows the secret key for honestly generated certificates, it refers to a higher level of trust, as the 'forged' signatures may go undetected for longer, as they are transmitted amongst valid message/signature pairs. Consider the case where the CA signs an invalid public key, and uses a private key only they know to authenticate messages. The user would (honestly) be signing messages using their private key, but their certificate verification would fail on the receiver's end, which would perhaps cause it to be added to the rogue list. Alternatively, if the CA knows the user's private key, i.e. it is using the same private key as the user, a dishonestly authenticated message from the CA would look the same as an honestly authenticated message from the user. This breach of trust may go undetected for longer before the certificate is revoked. Furthermore, the CA would store a copy of the user's private key, increasing the attack surface.

To ensure that the user can communicate their key over an unprotected channel, a PKE is used. This requires the CA to own an encryption-decryption key pair in addition to its signing-verification key pair that is required to generate certificates. As our construction is based on Dilithium, a good candidate for the PKE would be Kyber as it comes from the same cryptographic suite of algorithms, called CRYSTALS, and hence sub routines of the implementations can be used both for verification/decryption or signing/encryption. However, as the encryption

<pre> 1: User U 2: $A \leftarrow \text{Expand}(\rho)$ 3: $s_U, e_U \leftarrow S_\eta^l \times S_\eta^k$ 4: $t_U \leftarrow As_U + e_U \bmod q$ 5: $c_U \leftarrow \text{Encr}((s_U, e_U), ek_C)$ 6: $\xrightarrow{c_U, U}$ 7: $(s_U, e_U) \leftarrow \text{Decr}(c_U, dk_C)$ 8: $t_U \leftarrow As_U + e_U \bmod q$ 9: $R_U \leftarrow t_U + As_C + e_C$ 10: $R_U \leftarrow R_U \bmod q$ 11: $cert_U \leftarrow \text{Enc}(R_U, U)$ 12: $\xleftarrow{\sigma, cert_U}$ 13: $\sigma \leftarrow \text{Sign}(cert_U, sk_C)$ 14: if $\text{Vrfy}(\sigma, cert_U, vk_C) = 1$: $sk_U \leftarrow (s_U, e_U)$ $vk_U \leftarrow t_U$ </pre>	CA C
--	--------------------------

FIGURE 6. Dilithium-based construction assuming a trusted CA and a signature as well as an encryption key pair, reusing CA's public parameter ρ .

scheme is used in a black-box way, any PKE would do. We denote the encryption scheme by $\mathcal{E} = (\text{KGen}_{\mathcal{E}}, \text{Encr}, \text{Decr})$ and depict our construction in Figure 6.

Under initial inspection, this construction resembles an explicit certificate scheme, in the sense that the CA binds a public value to the user identity to create the certificate, and signs this. However, unlike in explicit certificate schemes, only the user needs to verify the CA's signature over the certificate, not the third-party verifier. The reason for this is the public value used in the certificate is the reconstruction value, rather than the entire public key. So, as per the definition of implicit certificates, the verifier uses R_U to reconstruct the public key of the user, using the CA's public key. In doing so, they should be convinced that pk_U is valid, as they only retrieve the correct public key $pk_U = t_U$ if a valid e_C and s_C have been used in the creation of R_U , and these secret values are only known by the CA.

We assume the CA has two key pairs: a Dilithium signing-verifying key pair $sk \leftarrow (\rho, t_C, s_C, e_C)$ and $pk \leftarrow (\rho, t_C)$ and an encryption-decryption key pair $(ek_C, dk_C) \leftarrow \text{KGen}_{\mathcal{E}}()$. When the user requests a certificate, they generate their own M-LWE instance, using the same A from the CA's instance, and encrypt the secret terms (s_U, e_U) to the CA. The CA uses knowledge of both their own and the user's secrets to create the reconstruction value, which resembles an M-LWE instance combining both parties' secrets, then encodes this reconstruction value with the user's identity U to produce the certificate. The CA signs this with their secret signing key. Upon receipt of the signed certificate, the user can verify the CA's signature. If the CA's signature is valid, they will set (s_U, e_U) as their secret and the corresponding t_U as their public key. The verifier

uses the CA's public key together with the reconstruction value to obtain the user's public key. More concretely, the verifier computes $vk_U \leftarrow R_U - t_C$. Correctness follows from the following equation:

$$\begin{aligned}
 vk_U = t_U &= As_U + e_U \\
 &= As_U + e_U + As_C + e_C - As_C - e_C \\
 &= A(s_C + s_U) + e_U + e_C - (As_C + e_C) \\
 &= R_U - t_C.
 \end{aligned}$$

Unfortunately, also this variant is not secure because, as we now demonstrate, the user can generate their own certificate. All they need is the CA's public key $pk_C = (A, t_C)$. As per the protocol, the user generates their own Dilithium key pair. However, rather than interacting with the CA, they can also generate and publish their own R_U by computing

$$R_U = A \cdot s_U + e_U + t_C.$$

The verifier derives t_U , oblivious to the fact that R_U was not generated using the CA's secret key. To mitigate this attack, we would need to pass the job of verifying the CA's signature on the certificate to the verifier. However, this would mean sending them the signature, i.e. an explicit certificate.

We now discuss some further properties of this construction that might be of interest nevertheless. Firstly, the use of two schemes means there is a requirement to store and distribute two CA public keys. If, for example, we chose to instantiate NIST Level 2 parameter sets for each scheme, the combined CA public key size would be greater than 2000 kB. We considered whether we could streamline the CA's keys, perhaps by using the same key for both signing and encryption. This could be possible as both Dilithium and Kyber are reliant upon the M-LWE problem, and so their keys are of similar format. The advantage is that only one key needs to be generated for the CA, leading to a smaller root certificate. Unfortunately, upon inspection of the parameter sets of each of the schemes, this would be decidedly impractical. Dilithium requires a much larger modulus than Kyber. More concretely, the modulus used in Dilithium is about 8×10^6 for all security levels, whilst Kyber requires a modulus of only 3×10^3 . If we increased the modulus to meet Dilithium's requirements, we would have to use larger dimension for Kyber to maintain the desired security level, and this would require increasing k from 2 to 4 (see Table 1) and a 2-fold increase of the ciphertext size.

The biggest disadvantage of this construction is of course that the CA knows sk_U . For example, [17] explicitly cover the case that the CA can be dishonest. A further disadvantage is the increased attack surface as the user secret keys are known by two parties, and this can be a security risk such as in the Trustico

breakthrough. With this paper we intend to stimulate further research on this topic and invite others to consider constructions that differ more substantially from the ECQV construction principles. For example, to construct a variant of Falcon's trapdoor sampling that would allow to decrease the signature size further, or turn to alternative schemes like identity-based or certificate-less signatures.

SUPPLEMENTARY MATERIAL

Supplementary material is available at www.comjnl.oxfordjournals.org.

DATA AVAILABILITY

The data underlying this article are available in the article and in its online supplementary material.

ACKNOWLEDGEMENTS

We would like to thank Dan Brown and Matthew Campagna for clarifications on the ECDSA–ECQV security, John Schanck for insightful discussion on the security of NTRU, and Johannes Braun for conversations on the definition of (public-key) certificates, and Paulo Barreto for initial conversations about implicit certificates from fiat-Shamir lattice-based signatures.

FUNDING

University of Waterloo Institute for Quantum Computing; IQC is supported in part by the Government of Canada through Innovation, Science and Economic Development Canada (ISED) and the Province of Ontario. In addition, N.B. was supported by Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery grant RGPIN-2016-05146, NSERC Discovery Accelerator Supplement grant RGPIN-2016-05146, and Contract 2L 165-180499/001/sv, "PQC Analysis", funded by Public Works and Government Services Canada. Moreover, S.M. was supported by Canada's NSERC Alliance Program and Public Works Government Services Canada (PWGSC).

REFERENCES

- [1] NIST (2016) Post-quantum crypto project. In <https://csrc.nist.gov/Projects/post-quantum-cryptography/Post-Quantum-Cryptography-Standardization>. National Institute of Standards and Technology, Gaithersburg, Maryland, United States.
- [2] Ducas, L., Lyubashevsky, V. and Prest, T. (2014) Efficient identity-based encryption over NTRU lattices. In *International Conf. Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan*. 7–11 December, pp. 22–41. Springer, New York, NY, United States.
- [3] Bert, P., Fouque, P.-A., Roux-Langlois, A. and Sabt, M. (2018) Practical implementation of ring-SIS/LWE based signature and IBE. In *Int. Conf. Post-Quantum Cryptography, Fort Lauderdale, Florida, United States*. 9–11 April, pp. 271–291. Springer, New York, NY, United States.
- [4] Gentry, C. (2009) Fully homomorphic encryption using ideal lattices. In *Proc. 41st Annual ACM Symposium on Theory of Computing, Bethesda, Maryland, United States*. May 31–June 2, pp. 169–178. Association for Computing Machinery, New York, NY, United States.
- [5] Stehlé, D. and Steinfeld, R. (2010) Faster fully homomorphic encryption. In *Int. Conf. Theory and Application of Cryptology and Information Security, Singapore, Singapore*. 5–9 December, pp. 377–394. Springer, New York, NY, United States.
- [6] Prest, T., Fouque, P.-A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Ricosset, T., Seiler, G., Whyte, W., and Zhang, Z. (2020) FALCON. *Technical report*. National Institute of Standards and Technology, Gaithersburg, Maryland, United States. Available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>.
- [7] Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., and Bai, S. (2020) CRYSTALS-DILITHIUM. *Technical report*. National Institute of Standards and Technology, Gaithersburg, Maryland, United States. Available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>.
- [8] NISTIR-8413 (2022). *Status report on the third round of the NIST post-quantum cryptography standardization process*. <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413.pdf>, National Institute of Standards and Technology, Gaithersburg, Maryland, United States.
- [9] Brown, D.R., Gallant, R. and Vanstone, S.A. (2001) Provably secure implicit certificate schemes. In *Int. Conf. Financial Cryptography, Grand Cayman, British West Indies*. 19–22 February, pp. 156–165. Springer, New York, NY, United States.
- [10] Qu, M. and Vanstone, S. Some new efficient implicit certificate schemes. *Preprint*. <https://dl.acm.org/doi/abs/10.5555/647505.725865#sec-ref>.
- [11] Pintsov, L.A. and Vanstone, S.A. (2000) Postal revenue collection in the digital age. In *Int. Conf. Financial Cryptography, Anguilla, British West Indies*. 21–24 February, pp. 105–120. Springer, New York, NY, United States.
- [12] IEEE Std. 1363a-2004 (2004) IEEE standard specifications for public-key cryptography - amendment 1: Additional techniques. In *Standard*. Institute of Electrical and Electronics Engineers, Piscataway, New Jersey, United States.
- [13] ISO/IEC 9796-3:2006 (2006) Information technology - security techniques - digital signature schemes giving message recovery - part 3: Discrete logarithm based mechanisms. In *Standard*. International Organization for Standardization, Geneva, CH.
- [14] ANSI X9.92-1-2009 (R2017) (2017) In Brown, D. (ed) *Public key cryptography for the financial services industry - digital signature algorithms giving partial message recovery - part 1: Elliptic curve pintsov-vanstone signatures (ecpvs)* Standard, Accredited Standards Committee X9, New York, NY, United States.
- [15] Qu, M. (2000) In Brown, D., Campagna, M., Vanstone, S. (eds) *Attack on ECDSA with original optimal mail certificates*.

- Personal communication between author. <https://eprint.iacr.org/2009/620.pdf>.
- [16] Kravitz, D. (2010) In Brown, D., Campagna, M., Vanstone, S. (eds) *Attack on ECQV-certified ECDSA*. Personal communication between author.
 - [17] Boldyreva, A., Fischlin, M., Palacio, A. and Warinschi, B. (2007) A closer look at PKI: security and efficiency. In *International Workshop on Public Key Cryptography, Beijing, China*. 16–20 April, pp. 458–475. Springer, New York, NY, United States.
 - [18] Brown, D. R. L., Campagna, M. J., and Vanstone, S. A. (2009). Security of ECQV-certified ECDSA against passive adversaries. Cryptology ePrint Archive, Report 2009/620. <https://eprint.iacr.org/2009/620>.
 - [19] Certicom Research: SEC 4 (2013) *Elliptic curve qu-vanstone implicit certificate scheme (ECQV)*. Technical report. Certicom Research, Mississauga, Canada.
 - [20] Liu, Z.-Y., Tseng, Y.-F., Tso, R., Wang, P.S. and Su, Q.-W. (2022) Extension of elliptic curve qu-vanstone certificates and their applications. *J. Inform. Security Appl.*, 67, 103176.
 - [21] IEEE Std. 1609.2 (2016) *IEEE standard for wireless access in vehicular environments-security services for applications and management messages*. Standard. Institute of Electrical and Electronics Engineers, Piscataway, New Jersey, United States.
 - [22] (2020) *IEEE standard for wireless access in vehicular environments (WAVE)-certificate management interfaces for end entities*. Standard. Institute of Electrical and Electronics Engineers, IEEE Std. 1609.2.1, Piscataway, New Jersey, United States.
 - [23] IEEE Std. 802.11p-2010 (2010) *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 6: Wireless Access in Vehicular Environments*. Standard. Institute of Electrical and Electronics Engineers, Piscataway, New Jersey, United States.
 - [24] Ha, D.A., Nguyen, K.T. and Zao, J.K. (2016) Efficient authentication of resource-constrained iot devices based on ecqv implicit certificates and datagram transport layer security protocol. In *Proc. 7th Symposium on Information and Communication Technology, Ho Chi Minh, Vietnam*. 8–9 December, pp. 173–179. Association for Computing Machinery, New York, NY, United States.
 - [25] Bindel, N., McCarthy, S., Twardokus, G., and Rahbari, H. (2022). *Drive (quantum) safe! - towards post-quantum security for v2v communications*. *Cryptology ePrint Archive*, Paper 2022/483. <https://eprint.iacr.org/2022/483>.
 - [26] Goldwasser, S., Micali, S. and Rivest, R.L. (1988) A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17, 281–308.
 - [27] Pornin, T. and Stern, J.P. (2005) Digital signatures do not guarantee exclusive ownership. In *Int. Conf. Applied Cryptography and Network Security, New York, NY, USA*. 7–10 June, pp. 138–150. Springer, New York, NY, United States.
 - [28] Jackson, D., Cremers, C., Cohn-Gordon, K. and Sasse, R. (2019) Seems legit: automated analysis of subtle attacks on protocols that use signatures. In *Proc. 2019 ACM SIGSAC Conf. Computer and Communications Security, London, UK*. 11–15 November, pp. 2165–2180. Association for Computing Machinery, New York, NY, United States.
 - [29] Brendel, J., Cremers, C., Jackson, D., and Zhao, M. (2020). *The provable security of Ed25519: theory and practice*. *Cryptology ePrint Archive, Report 2020/823*. <https://eprint.iacr.org/2020/823>.
 - [30] Blake-Wilson, S. and Menezes, A. (1999) Unknown key-share attacks on the station-to-station (sts) protocol. In *Int. Workshop on Public Key Cryptography, Kamakura, Japan*. 1–3 March, pp. 154–170. Springer, New York, NY, United States.
 - [31] Menezes, A. and Smart, N.P. (2004) Security of signature schemes in a multi-user setting. *Des. Codes Cryptogr.*, 33, 261–274.
 - [32] Cremers, C., Düzlül, S., Fiedler, R., Fischlin, M. and Janson, C. (2021) Buffing signature schemes beyond unforgeability and the case of post-quantum signatures. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24–27 May 2021*, pp. 1696–1714. IEEE, Piscataway, New Jersey, United States.
 - [33] Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R. and Polk, W. (2008) *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280 (Proposed Standard)*. IETF.
 - [34] Bindel, N., Herath, U., McKague, M. and Stebila, D. (2017) Transitioning to a quantum-resistant public key infrastructure. In *Int. Workshop on Post-Quantum Cryptography, Daejeon, Korea*. 15–17 April, pp. 384–405. Springer, New York, NY, United States.
 - [35] Kiltz, E. and Neven, G. (2008) Identity-based signatures. In Marc, J., Neven, G. (eds) *Cryptology and information security series*. 2, pp. 31–44. <https://doi.org/10.3233/978-1-58603-947-9-31>.
 - [36] Chuengsatiansup, C., Prest, T., Stehlé, D., Wallet, A. and Xagawa, K. (2020) Modfalcon: compact signatures based on module-ntru lattices. In *Proc. 15th ACM Asia Conf. Computer and Communications Security, Taipei, Taiwan*. 5–9 October, pp. 853–866. Association for Computing Machinery, New York, NY, United States.
 - [37] liboqs sig performance. https://openquantumsafe.org/benchmarking/visualization/speed_sig.html (Accessed January 1, 2022).
 - [38] Fiat, A. and Shamir, A. (1986) How to prove yourself: practical solutions to identification and signature problems. In *Conf. Theory and Application of Cryptographic Techniques, Santa Barbara, CA, United States*. 11–15 August, pp. 186–194. Springer, New York, NY, United States.
 - [39] Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Seiler, G., and Stehlé, D. (2020) CRYSTALS-KYBER. Technical report. National Institute of Standards and Technology, Gaithersburg, Maryland, United States. Available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>.